

title: Thiết lập config KZTEK dùng chung cho mọi project

created: 2026-07-25

updated: 2026-07-25

Config KZTEK dùng chung — Cài đặt & Vận hành

Vấn đề đã giải quyết: Trước đây mỗi project muốn dùng bộ agent/skill/template KZTEK đều phải **copy tay** thư mục `.claude/` sang. Nhiều project → nhiều bản sao → lệch phiên bản, sửa một chỗ không lan sang chỗ khác.

>

Cách giải quyết: Đưa config lên **user-level scope** (`~/.claude`) bằng junction trở về repo này. Một nguồn duy nhất, git quản lý đầy đủ, mọi project trên máy tự động dùng được.

1. Nguyên lý

Claude Code đọc config theo 3 tầng, hợp nhất tự động:

Tầng	Đường dẫn	Phạm vi
---	---	---
User (global)	<code>C:\Users\<user>\.claude\</code>	Mọi project trên máy
Project	<code><project>\.claude\</code>	Một project
Local	<code><project>\.claude\settings.local.json</code>	Một project, không commit

Junction là **thư mục giả** ở tầng User, chuyển tiếp mọi truy cập về file thật trong repo:

```
C:\Users\nguye\.claude\agents      ← junction (~0 byte)
    | mọi truy cập chuyển tiếp tới ↓
Desktop\Claude-Git\claude\.claude\agents ← file thật, git quản lý
```

Chỉ có **một** bộ file. Sửa `~/.claude/agents/cto.md` == sửa file trong repo == `git status` thấy ngay.

Junction ≠ Shortcut (`.lnk`): shortcut chỉ Explorer hiểu, chương trình khác coi là file lạ.

Junction hoạt động ở tầng hệ thống file nên Claude Code / python / git đều xem như thư mục thật.

2. Cài đặt trên máy mới

```
git clone https://github.com/flick-git-anhvn/claude.git C:\Users\<user>\Desktop\Claude-Git\claude
cd C:\Users\<user>\Desktop\Claude-Git\claude
powershell -ExecutionPolicy Bypass -File scripts\link-global.ps1
```

Không cần quyền admin. Sau đó merge 2 mục trong `.claude/templates/settings-global.json` vào `~/.claude/settings.json` — **merge, không copy đè** (file đó có thể đã chứa `model`, `theme`, `enabledPlugins`, hook riêng của máy).

Kiểm tra:

```
powershell -ExecutionPolicy Bypass -File scripts\link-global.ps1 -WhatIf # xem trước, không ghi
Get-ChildItem C:\Users\<user>\.claude -Force | Where-Object { $_.LinkType -eq 'Junction' }
```

3. Bảng 8 junction

Junction tại <code>~/ .claude\`</code>	Nguồn trong repo	Nội dung
---	---	---
agents	.claude/agents	19 agent (kèm agents/references/)
commands	.claude/commands	Skill / slash command
shared	.claude/shared	CORE.md, GOTCHAS.md
templates	.claude/templates	PRD, TDD, PLAN, EVAL...
references	.claude/references	Checklist bảo mật, DoD
evals	.claude/evals	Eval của agent/skill
hooks-kztek	.claude/hooks	config-protection.js
scripts	scripts (repo root)	md_to_docx_kztek.py, linux-deb, kiosk...

Vì sao tên ``hooks-kztek`` mà không phải ``hooks``: `~/ .claude/hooks/` đã có hook riêng của máy (`code-graph-lesson-reminder.js`) không thuộc repo này. Dùng tên khác để hai bộ cùng tồn tại.

Vì sao KHÔNG junction ``CLAUDE.md``: quy trình 17-agent / Dispatcher / plan file chỉ phù hợp project phần mềm, không nên áp lên mọi thư mục. Project nào cần thì tự có `CLAUDE.md` riêng, hoặc nạp bằng một dòng `@C:/Users/nguye/Desktop/Claude-Git/claude/CLAUDE.md`.

4. ⚠ Cấm tuyệt đối: junction vào trong working tree của project khác

Git đi xuyên junction và coi nó là thư mục thật. Nếu tạo

`<projectB>\.claude\agents` → trở vào repo này, thì `git add -A` ở project B sẽ hút

toàn bộ file của repo config vào repo B.

Chỉ junction tại `~/ .claude` — nằm ngoài mọi working tree, git không bao giờ quét tới.

5. Quy tắc đường dẫn trong file config

Vì agent giờ được load từ **mọi project**, đường dẫn tương đối không còn tin được.

Loại	Cách viết	Ví dụ
---	---	---
Hạ tầng dùng chung	Tuyệt đối	C:/Users/nguye/.claude/templates/PRD-template.md
Sản phẩm của project	Tương đối	docs/plans/, src/, code-graph/, _workspace/

Sai hướng nào cũng lỗi: hạ tầng viết tương đối → không tìm thấy ở project khác;

sản phẩm viết tuyệt đối → project A ghi đè output của project B.

Script `md_to_docx_kztek.py` tự phân giải logo theo `Path(__file__).resolve()` (đi xuyên junction ra đường dẫn thật trong repo), nên tài liệu xuất từ project nào cũng có logo KZTEK.

Trước khi sửa, hàm `find_logo()` chỉ tìm theo CWD → chạy từ project khác là **mất logo mà không báo lỗi**.

6. Vận hành hằng ngày

Việc	Cách làm
---	---
Sửa agent/skill/template dùng chung	Sửa trực tiếp (qua <code>~/.claude/...</code> hoặc trong repo — cùng một file)
Commit thay đổi đó	<code>/sync-global</code> — chạy được từ bất kỳ project nào
Lấy bản mới nhất về máy	<code>/sync-global</code> (bước pull)
Di chuyển repo sang ổ/thư mục khác	Chạy lại <code>link-global.ps1</code> — script tự phát hiện junction trở sai và tạo lại
Xuất DOCX/PDF từ project bất kỳ	<code>python</code> <code>C:/Users/nguye/.claude/scripts/md_to_docx_kztek.py</code> <code><file.md></code>

7. Rollback

```
# Bỏ toàn bộ junction — file thật trong repo KHÔNG bị ảnh hưởng
powershell -ExecutionPolicy Bypass -File scripts\link-global.ps1 -Unlink

# Trả settings.json / commands / hooks về bản trước khi đổi
Copy-Item C:\Users\nguye\.claude\_backup-before-junction-2026-07-25\*
C:\Users\nguye\.claude\ -Recurse -Force

# Trả code trong repo về trạng thái cũ
git -C <repo> switch main
```

Xoá junction bằng `rmdir` **không** xoá nội dung đích — đây là hành vi của junction, không phải may mắn.

Script `-Unlink` dùng `cmd /c rmdir` chính vì lý do này (`Remove-Item -Recurse` trên junction từng có bug xoá xuyên sang target ở PowerShell cũ).

8. Ghi chú về git

- File `.docx`/`.pdf` của tài liệu hạ tầng đã bỏ tracking (`.claude/**`, `scripts/**`, `CLAUDE`, `RULES`, `WORKFLOW`). Lý do: §19 CLAUDE.md bắt xuất DOCX+PDF sau mỗi lần sửa `.md`; binary không delta được nên mỗi lần sửa `CLAUDE.md` cộng thêm ~1,2 MB vĩnh viễn vào `.git` (đã lên 162 MB cho 71 file text). Các file này sinh lại được bằng script. Deliverable trong `docs/` vẫn tracked bình thường.

- Junction **không** nằm trong git (là cấu hình máy, không phải nội dung).
Cách cài được version hoá qua `scripts/link-global.ps1` — clone xong chạy script là xong.

Lịch sử cập nhật

Ngày	Nội dung	Người
---	---	---
2026-07-25	Tạo mới — chuyển config sang user-level scope bằng 8 junction	Claude Code